COVID-19

A Single Line of Python Code Scraping Dataset from Webpages

Hunting for API endpoints from webpages and downloads using Python

Christopher Tao
May 31, 2020 • 6 min read

photo by Markus Spiske on Unsplash

No matter what level of data science/analytics skills we have, you cannot do anything without datasets.

Indeed, there are many open-source datasets such as Kaggle and Data.world. However, they are more suitable to be used for exercises and learning purposes, but may not satisfy our general needs.

Usually, data scientists/analysts may have more or less web scraping skills, so it will be much easier to get datasets whenever you saw on the websites. After scraping the content from the websites, a series of transforming, extracting and cleansing manipulations will help us to get the clean dataset for the next step. This is one of the typical usages of Python because there are many excellent web scraping libraries available in Python such as Scrapy and Beautiful Soup.

However, learning these web scraping libraries is not going to be that easy. Also, for those who do not have web development backgrounds, which is probably not a common skill for data scientists/analysts, it might be even more difficult to understand and nail the concepts such as HTML DOM and XPath.

However, in this article, I will introduce a much easier way of downloading datasets from websites. You will find that you can even use Pandas to do all the work!

COVID-19 New Cases Dataset

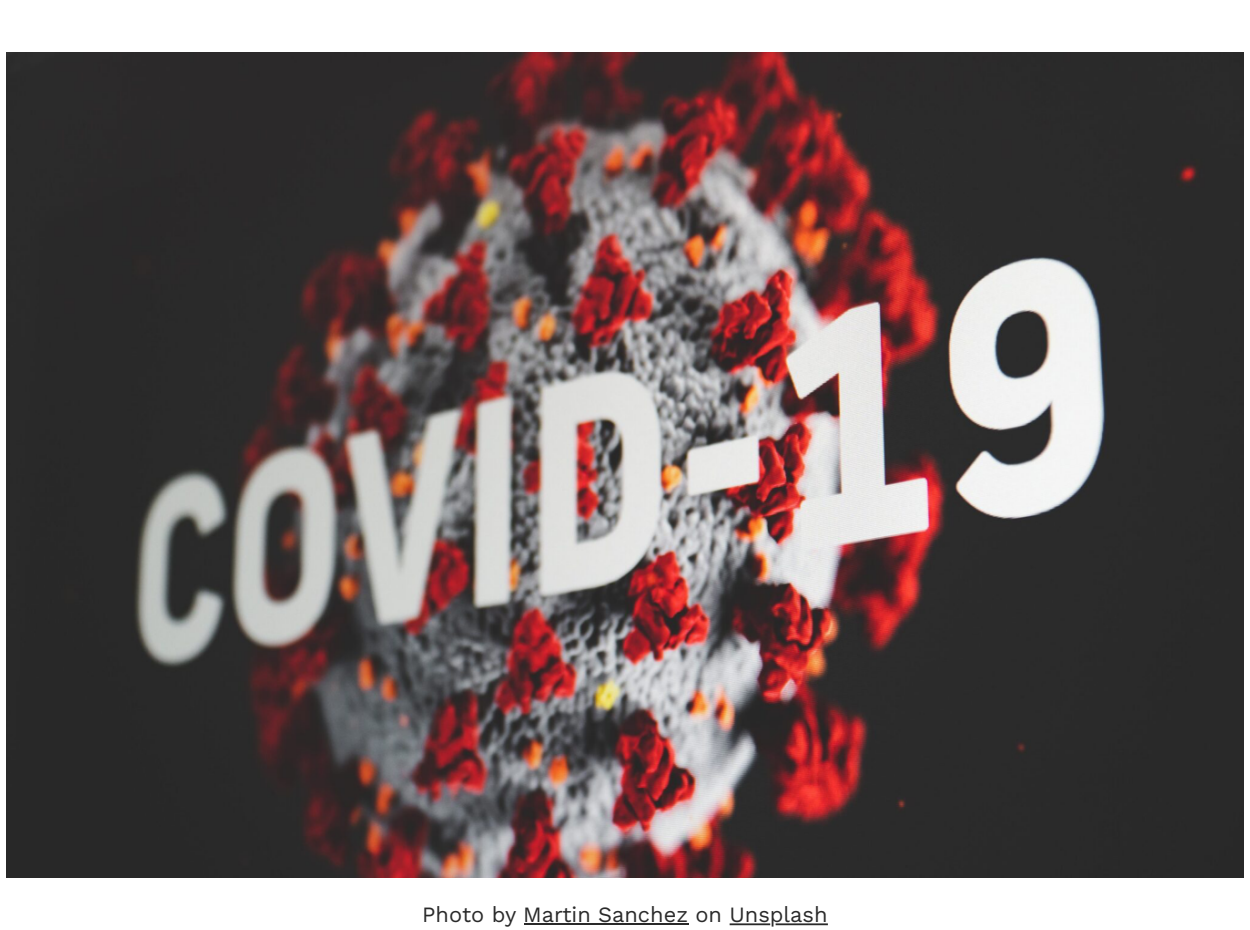
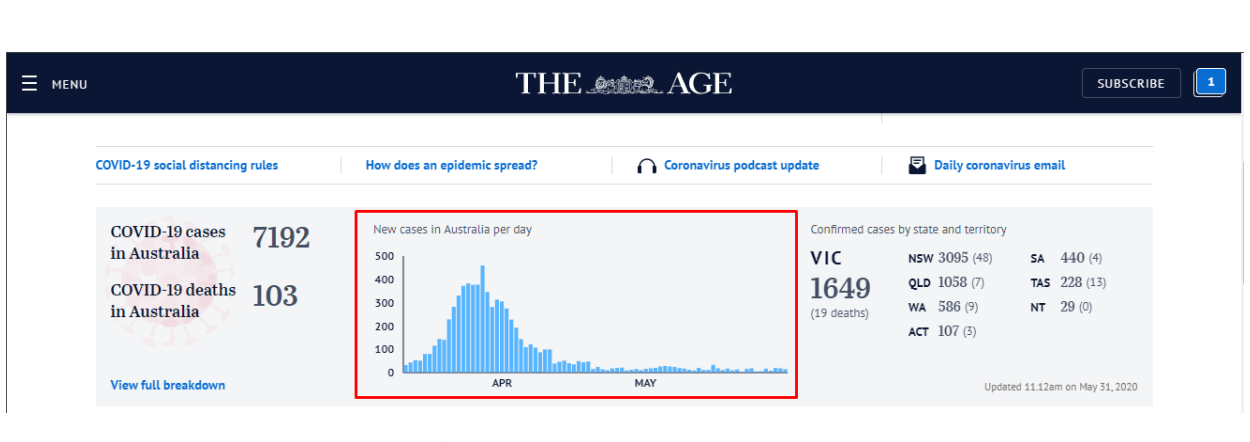


Photo by Martin Sanchez on Unsplash

The first example is the Covid-19 New Cases dataset. It turns out there are many publicly available data sources such as the government websites which will publish this. However, I just want to use this as an example, as I found that this is a very good and typical example.

Today, when I scan news from "The Age" (A popular media of Australia) website, I found that there is a bar chart showing daily new confirmed cases in Australia (see screenshot below).


MENU
THE AGE
SUBSCRIBE
COVID-19 social distancing rules
How does an epidemic spread?
Coronavirus podcast update
Daily coronavirus email
New cases in Australia per day
APR
MAY
View full breakdown

COVID-19 cases in Australia 7192
COVID-19 deaths in Australia 103

Confirmed cases by state and territory
VIC 1649 (19 deaths)
NSW 3095 (48)
QLD 1058 (7)
WA 586 (9)
ACT 107 (3)
SA 440 (4)
TAS 228 (13)
NT 29 (0)
Updated 11:12am on May 31, 2020

Screenshot from the website: https://www.theage.com.au/

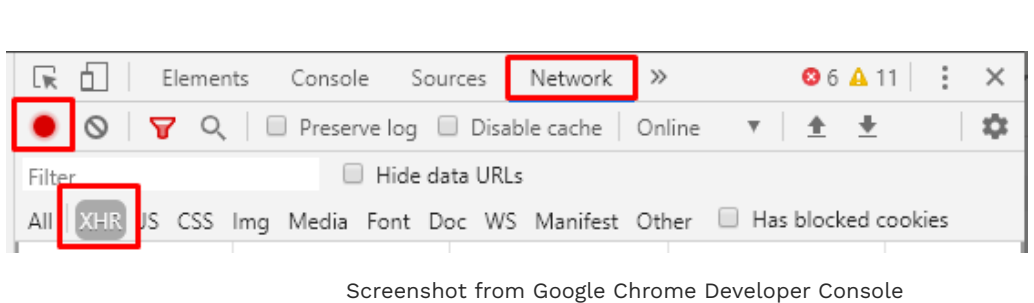
What if I want to get their Data? Let's find its API endpoint!

Hunt for API Endpoint

Most of the popular web browsers provide their developer consoles now. We will use the console to do this job. Here are some shortcuts to call out the console from your browsers:

- Google Chrome: Ctrl + Shift + I
- Firefox: Ctrl + Shift + K
- Microsoft Edge: F12

Here I will use Google Chrome for demonstration. But don't worry, most of the browsers have very similar developer consoles. I believe you will be able to figure out where are the features.



Screenshot from Google Chrome Developer Console

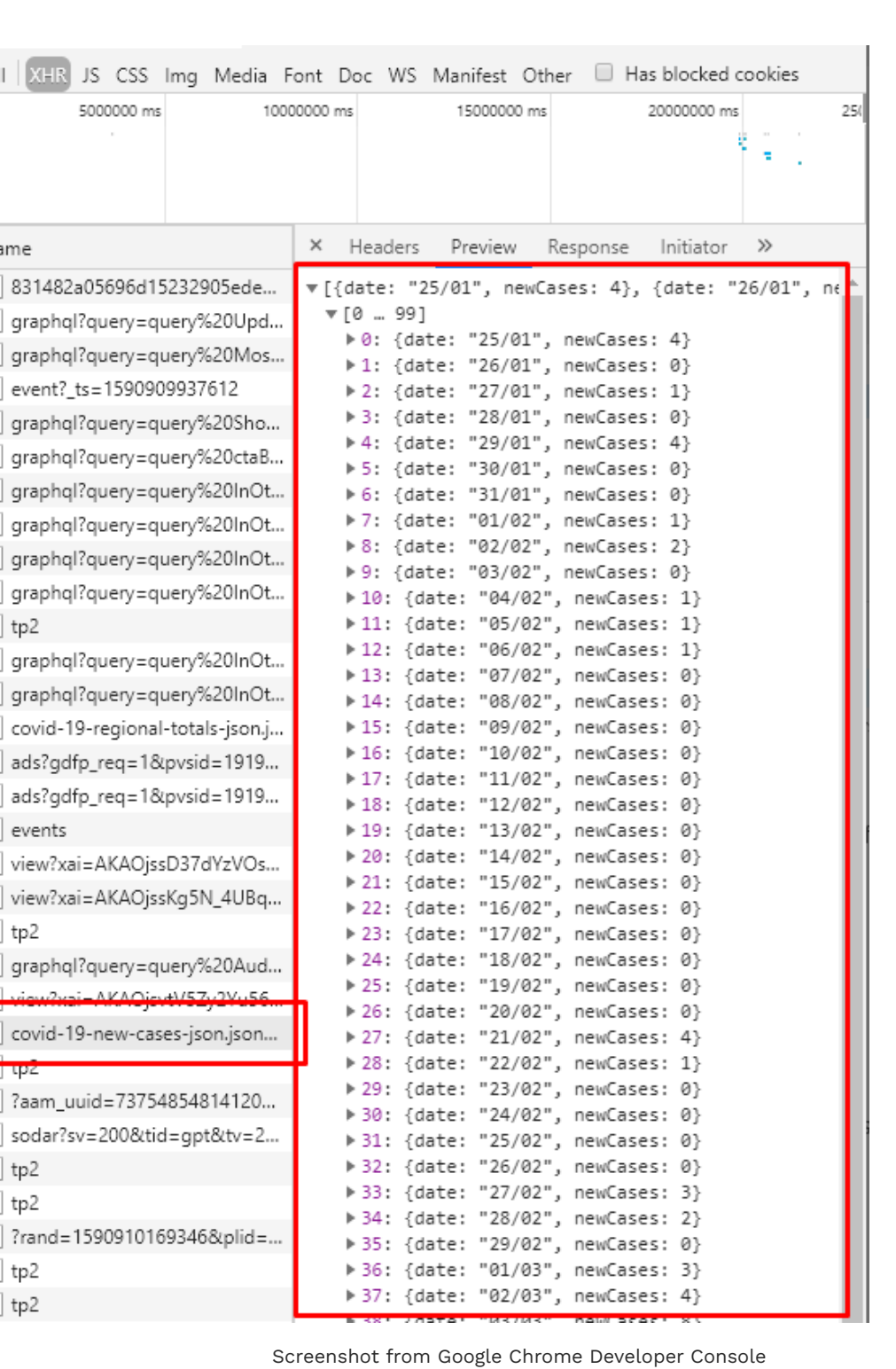
Once the console appears, go to the "Network" tab. We are looking for an API endpoint, so it would be caught as an "XHR" request if it is available. So, let's select the XHR filter.

> **XMLHttpRequest** (**XHR**) is an API in the form of an object whose methods transfer data between a web browser and a web server. The object is provided by the browser's JavaScript environment. Particularly, retrieval of data from XHR for the purpose of continually modifying a loaded web page is the underlying concept of Ajax design. Despite the name, XHR can be used with protocols other than HTTP and data can be in the form of not only XML, but also JSON, HTML or plain text. [1]

Sometimes, the webpage may have some JavaScripts running in the background to do some scheduled jobs and the browser will catch them all. If you want to have a cleaner list of caught requests, the best approach is to

- Make sure the "recording" button is enabled
- Refresh the webpage
- Stop "recording" when you see the data related content has already been fully rendered on the webpage

Now, you will have a list of requests in the developer console.



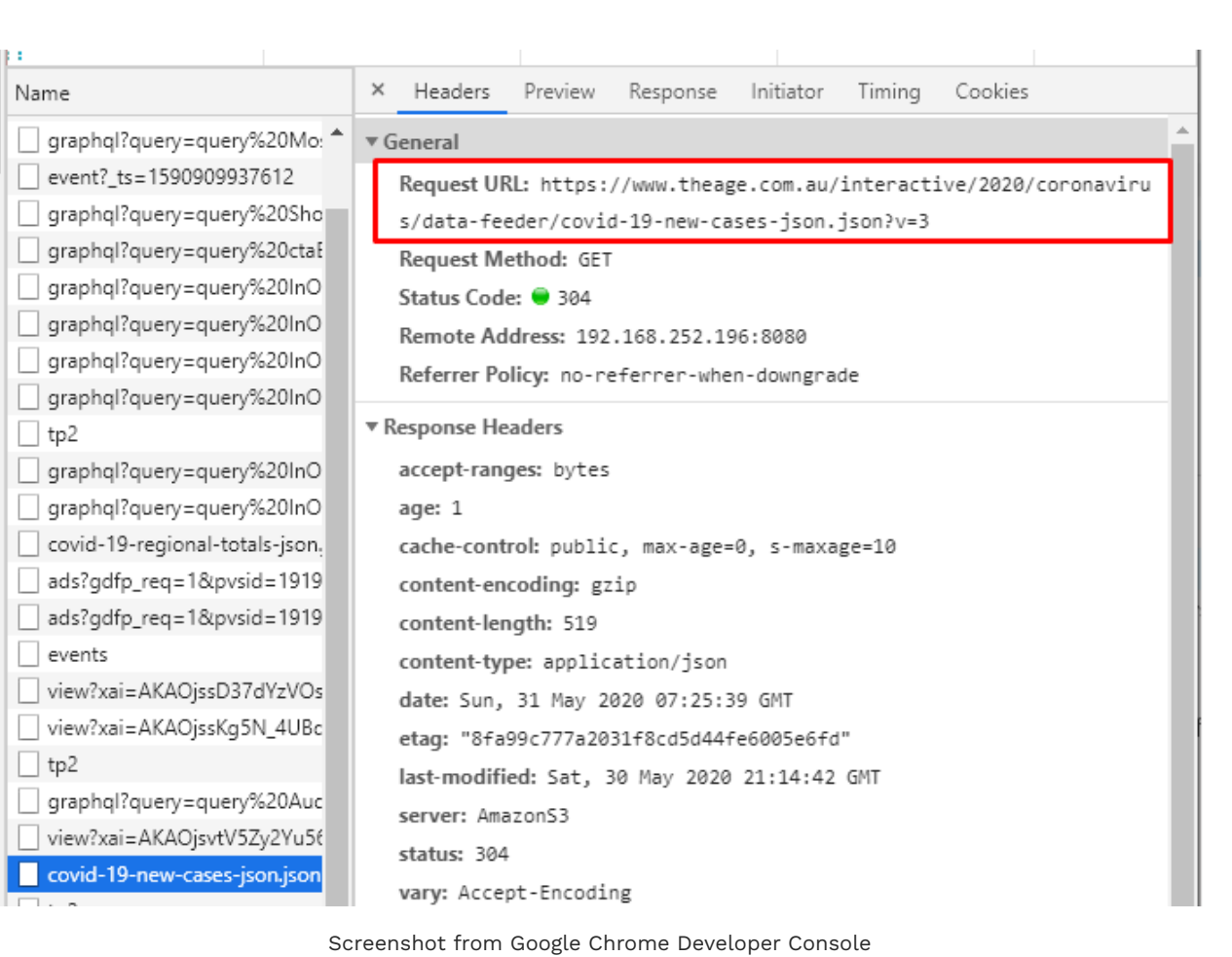
```
831482a05696d15232905ede...
graphql?query=query%20Upd...
graphql?query=query%20Mos...
event?_ts=1590909937612
graphql?query=query%20Sho...
graphql?query=query%20ctaB...
graphql?query=query%20InOt...
graphql?query=query%20InOt...
graphql?query=query%20InOt...
graphql?query=query%20InOt...
tp2
graphql?query=query%20InOt...
graphql?query=query%20InOt...
covid-19-regional-totals-json.j...
ads?gdfp_req=1&pvsid=1919...
ads?gdfp_req=1&pvsid=1919...
events
view?xai=AKAOjssD37dYzVOs...
view?xai=AKAOjssKg5N_4UBq...
tp2
graphql?query=query%20Aud...
covid-19-new-cases-json.json...
tp2
?aam_uuid=73754854814120...
sodar?sv=200&tid=gpt&tv=2...
tp2
tp2
?rand=1590910169346&plid=...
tp2
tp2
```

```
▼[{date: "25/01", newCases: 4}, {date: "26/01", ne
 ▼[0 … 99]
  ▶0: {date: "25/01", newCases: 4}
  ▶1: {date: "26/01", newCases: 0}
  ▶2: {date: "27/01", newCases: 1}
  ▶3: {date: "28/01", newCases: 0}
  ▶4: {date: "29/01", newCases: 4}
  ▶5: {date: "30/01", newCases: 0}
  ▶6: {date: "31/01", newCases: 0}
  ▶7: {date: "01/02", newCases: 1}
  ▶8: {date: "02/02", newCases: 2}
  ▶9: {date: "03/02", newCases: 0}
  ▶10: {date: "04/02", newCases: 1}
  ▶11: {date: "05/02", newCases: 1}
  ▶12: {date: "06/02", newCases: 1}
  ▶13: {date: "07/02", newCases: 0}
  ▶14: {date: "08/02", newCases: 0}
  ▶15: {date: "09/02", newCases: 0}
  ▶16: {date: "10/02", newCases: 0}
  ▶17: {date: "11/02", newCases: 0}
  ▶18: {date: "12/02", newCases: 0}
  ▶19: {date: "13/02", newCases: 0}
  ▶20: {date: "14/02", newCases: 0}
  ▶21: {date: "15/02", newCases: 0}
  ▶22: {date: "16/02", newCases: 0}
  ▶23: {date: "17/02", newCases: 0}
  ▶24: {date: "18/02", newCases: 0}
  ▶25: {date: "19/02", newCases: 0}
  ▶26: {date: "20/02", newCases: 0}
  ▶27: {date: "21/02", newCases: 4}
  ▶28: {date: "22/02", newCases: 1}
  ▶29: {date: "23/02", newCases: 0}
  ▶30: {date: "24/02", newCases: 0}
  ▶31: {date: "25/02", newCases: 0}
  ▶32: {date: "26/02", newCases: 0}
  ▶33: {date: "27/02", newCases: 3}
  ▶34: {date: "28/02", newCases: 2}
  ▶35: {date: "29/02", newCases: 0}
  ▶36: {date: "01/03", newCases: 3}
  ▶37: {date: "02/03", newCases: 4}
```

Screenshot from Google Chrome Developer Console

OK. This one is very obvious. We can see that there is a request to have its name called "covid-19-new-cases-json.json…". That must be it!

Go to the "Headers" tab, you will see the detail of this request. The most important thing is the URL.

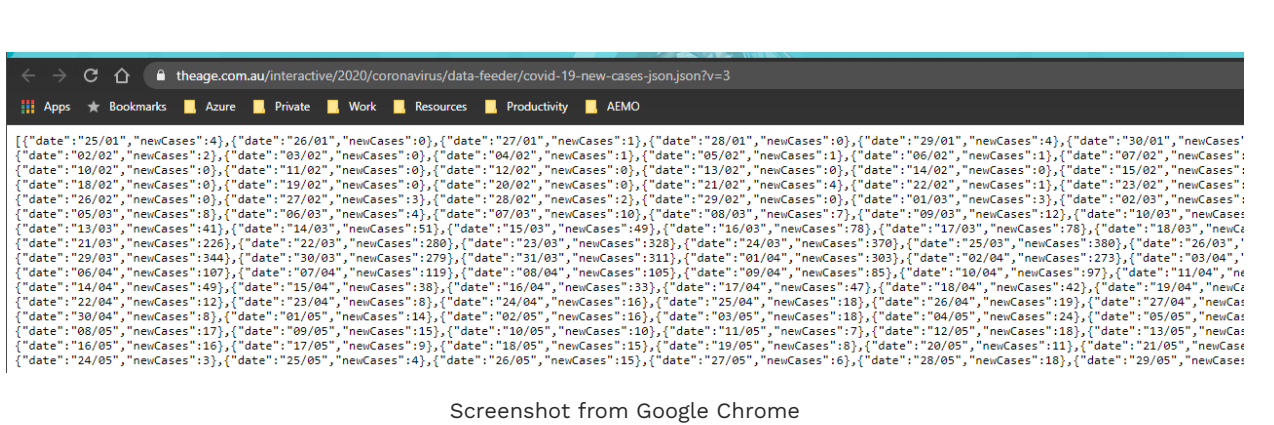


```
graphql?query=query%20Mo:
event?_ts=1590909937612
graphql?query=query%20Sho
graphql?query=query%20ctaE
graphql?query=query%20InO
graphql?query=query%20InO
graphql?query=query%20InO
graphql?query=query%20InO
tp2
graphql?query=query%20InO
graphql?query=query%20InO
covid-19-regional-totals-json.
ads?gdfp_req=1&pvsid=1919
ads?gdfp_req=1&pvsid=1919
events
view?xai=AKAOjssD37dYzVOs
view?xai=AKAOjssKg5N_4UBc
tp2
graphql?query=query%20Auc
view?xai=AKAOjsvtV5Zy2Yu5€
covid-19-new-cases-json.json
```

```
▼General
  Request URL: https://www.theage.com.au/interactive/2020/coronaviru
  s/data-feeder/covid-19-new-cases-json.json?v=3
  Request Method: GET
  Status Code: ● 304
  Remote Address: 192.168.252.196:8080
  Referrer Policy: no-referrer-when-downgrade
▼Response Headers
  accept-ranges: bytes
  age: 1
  cache-control: public, max-age=0, s-maxage=10
  content-encoding: gzip
  content-length: 519
  content-type: application/json
  date: Sun, 31 May 2020 07:25:39 GMT
  etag: "8fa99c777a2031f8cd5d44fe6005e6fd"
  last-modified: Sat, 30 May 2020 21:14:42 GMT
  server: AmazonS3
  status: 304
  vary: Accept-Encoding
```

Screenshot from Google Chrome Developer Console

Now, let's just open that URL as another browser tab to see what happen.


theage.com.au/interactive/2020/coronavirus/data-feeder/covid-19-new-cases-json.json?v=3
Apps
Bookmarks
Azure
Private
Work
Resources
Productivity
AEMO

Screenshot from Google Chrome

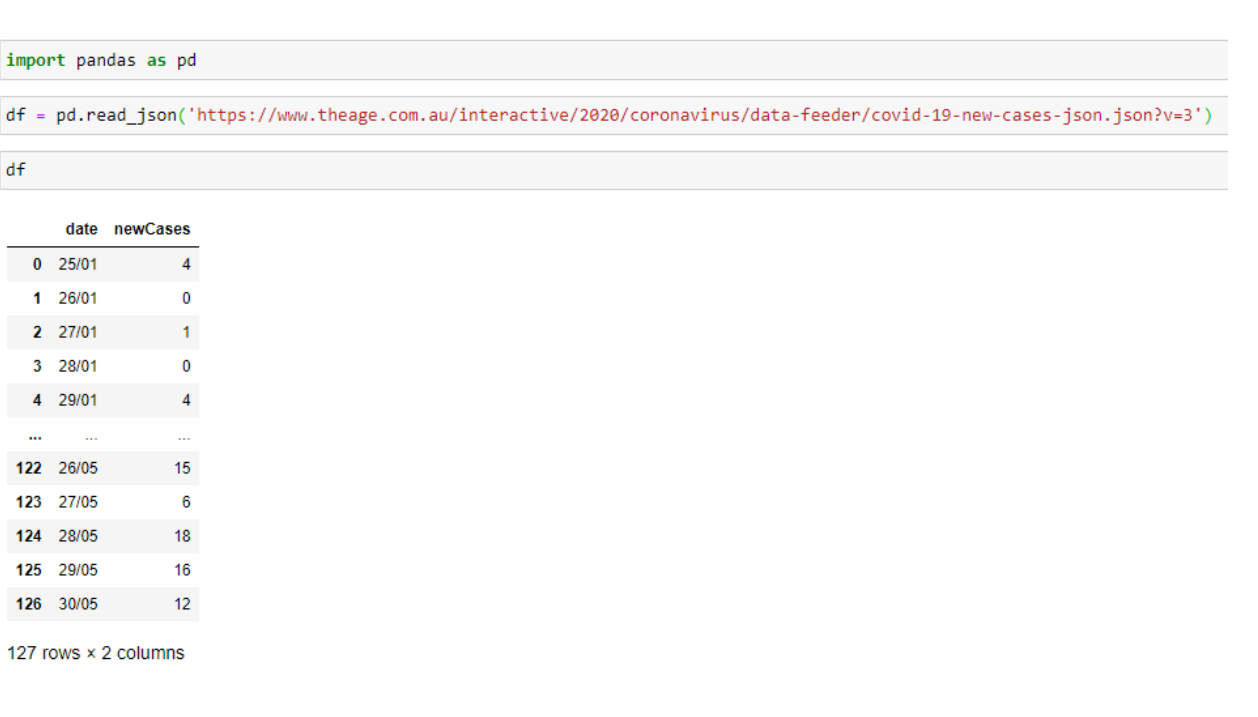
Cool! This is the API endpoint we're looking for.

Read API Endpoint using Pandas

How to consume it? Extremely easy!

I believe you must use Pandas Dataframe if you are a data scientist or data analyst. Just one line of Python code, you got everything directly into Pandas Dataframe.

```
df = pd.read_json('https://www.theage.com.au/interactive/2020/coronavirus/data-feeder/covid-19-new-cases-json.json?v=3')
```



```
import pandas as pd
```

```
df = pd.read_json('https://www.theage.com.au/interactive/2020/coronavirus/data-feeder/covid-19-new-cases-json.json?v=3')
```

```
df
```

	date	newCases
0	25/01	4
1	26/01	0
2	27/01	1
3	28/01	0
4	29/01	4
...	...	...
122	26/05	15
123	27/05	6
124	28/05	18
125	29/05	16
126	30/05	12

127 rows × 2 columns

IKEA Furniture list

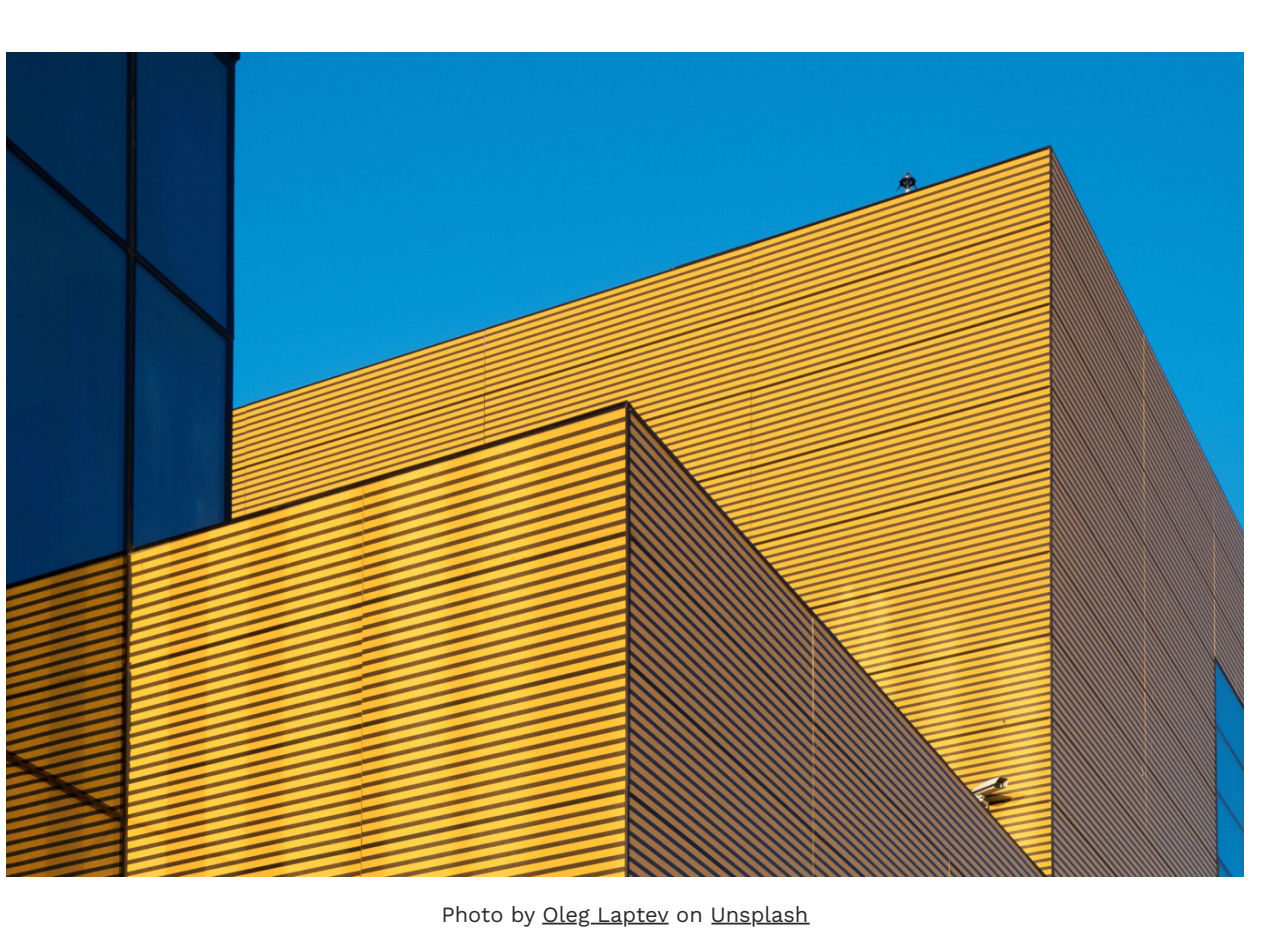
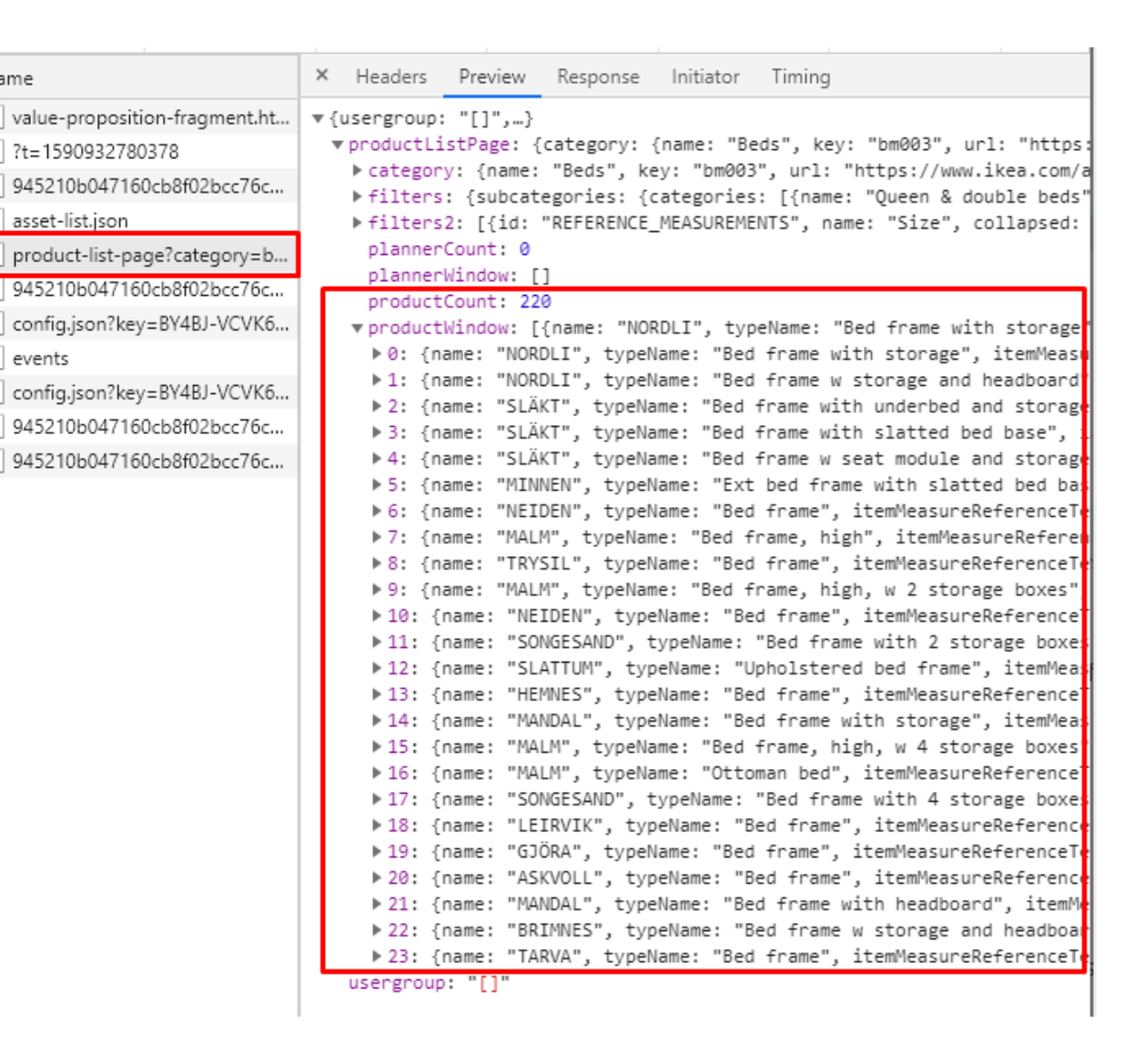


Photo by Oleg Laptev on Unsplash

Not all websites are using REST API endpoints so that I wouldn't say this is a universal approach. However, you'll find that there are a considerable number of websites using that. Let's just take one more example, the IKEA website.

Let's say you want to get all the beds from IKEA together with the product details to perform some analysis. Here is the request that my browser caught from the URL (https://www.ikea.com/au/en/cat/beds-bm003/).

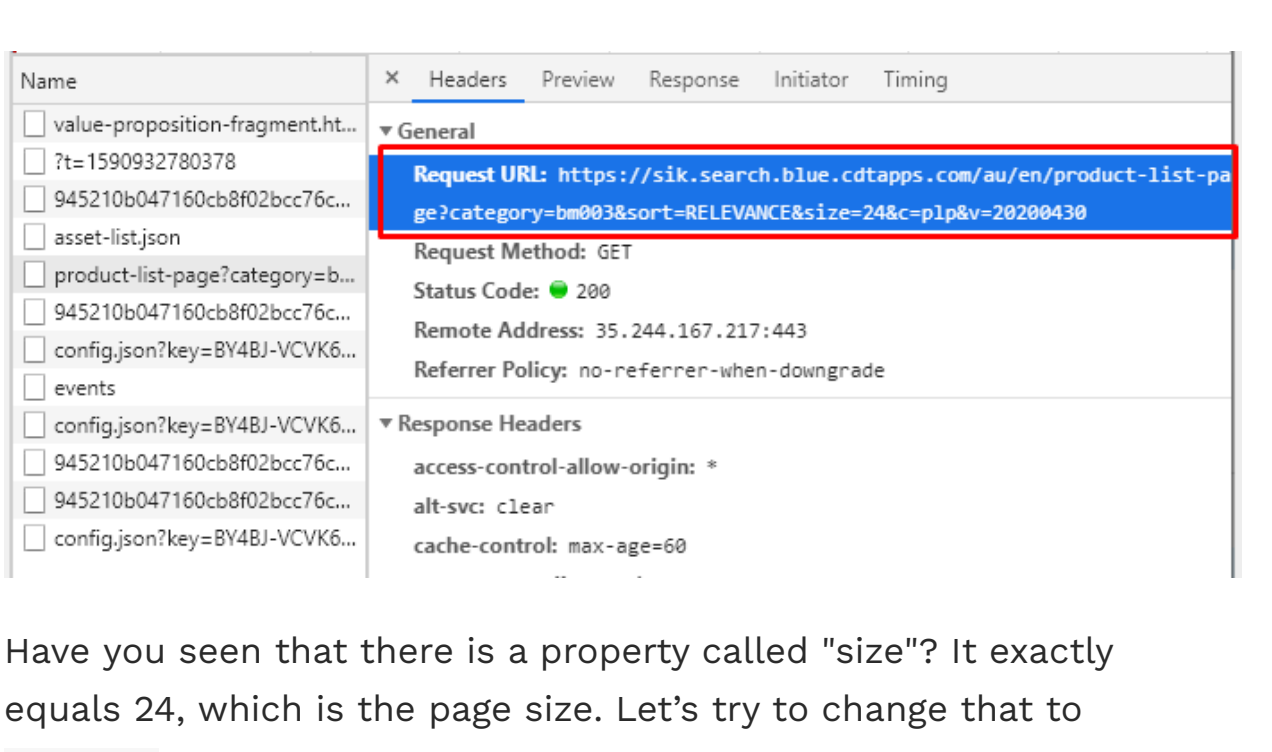


```
value-proposition-fragment.ht...
?t=1590932780378
945210b047160cb8f02bcc76c...
asset-list.json
product-list-page?category=b...
945210b047160cb8f02bcc76c...
config.json?key=BY4BJ-VCVK6...
events
config.json?key=BY4BJ-VCVK6...
945210b047160cb8f02bcc76c...
945210b047160cb8f02bcc76c...
```

```
▼{usergroup: "[]",…}
  ▼productListPage: {category: {name: "Beds", key: "bm003", url: "https:
    ▶category: {name: "Beds", key: "bm003", url: "https://www.ikea.com/a
    ▶filters: {subcategories: {categories: [{name: "Queen & double beds"
    ▶filters2: [{id: "REFERENCE_MEASUREMENTS", name: "Size", collapsed:
     plannerCount: 0
     plannerWindow: []
     productCount: 220
    ▼productWindow: [{name: "NORDLI", typeName: "Bed frame with storage"
      ▶0: {name: "NORDLI", typeName: "Bed frame with storage", itemMeas
      ▶1: {name: "NORDLI", typeName: "Bed frame w storage and headboard
      ▶2: {name: "SLÄKT", typeName: "Bed frame with underbed and storag
      ▶3: {name: "SLÄKT", typeName: "Bed frame with slatted bed base",
      ▶4: {name: "SLÄKT", typeName: "Bed frame w seat module and storag
      ▶5: {name: "MINNEN", typeName: "Ext bed frame with slatted bed ba
      ▶6: {name: "NEIDEN", typeName: "Bed frame", itemMeasureReferenceT
      ▶7: {name: "MALM", typeName: "Bed frame, high", itemMeasureRefere
      ▶8: {name: "TRYSIL", typeName: "Bed frame", itemMeasureReferenceT
      ▶9: {name: "MALM", typeName: "Bed frame, high, w 2 storage boxes"
      ▶10: {name: "NEIDEN", typeName: "Bed frame", itemMeasureReference
      ▶11: {name: "SONGESAND", typeName: "Bed frame with 2 storage boxe
      ▶12: {name: "SLATTUM", typeName: "Upholstered bed frame", itemMea
      ▶13: {name: "HEMNES", typeName: "Bed frame", itemMeasureReferenceT
      ▶14: {name: "MANDAL", typeName: "Bed frame with storage", itemMea
      ▶15: {name: "MALM", typeName: "Bed frame, high, w 4 storage boxes
      ▶16: {name: "MALM", typeName: "Ottoman bed", itemMeasureReference
      ▶17: {name: "SONGESAND", typeName: "Bed frame with 4 storage boxe
      ▶18: {name: "LEIRVIK", typeName: "Bed frame", itemMeasureReferenc
      ▶19: {name: "GJÖRA", typeName: "Bed frame", itemMeasureReferenceT
      ▶20: {name: "ASKVOLL", typeName: "Bed frame", itemMeasureReferenc
      ▶21: {name: "MANDAL", typeName: "Bed frame with headboard", itemM
      ▶22: {name: "BRIMNES", typeName: "Bed frame w storage and headboa
      ▶23: {name: "TARVA", typeName: "Bed frame", itemMeasureReferenceT
  usergroup: "[]"
```

This one is more interesting. Please note that it says that the "product count" is 220, but the "product window" only give 24 of them. If you pay attention to the page, it turns out only 24 products will be listed on the page, and a "load more" button is provided if you want to keep browsing more. Well, this is very common in Web Development to save bandwidth and server resources.

But does that mean we have to give it up? Absolutely no. Let's have a look at the request URL:



```
value-proposition-fragment.ht...
?t=1590932780378
945210b047160cb8f02bcc76c...
asset-list.json
product-list-page?category=b...
945210b047160cb8f02bcc76c...
config.json?key=BY4BJ-VCVK6...
events
config.json?key=BY4BJ-VCVK6...
945210b047160cb8f02bcc76c...
945210b047160cb8f02bcc76c...
config.json?key=BY4BJ-VCVK6...
```

```
▼General
  Request URL: https://sik.search.blue.cdtapps.com/au/en/product-list-pa
  ge?category=bm003&sort=RELEVANCE&size=24&c=plp&v=20200430
  Request Method: GET
  Status Code: ● 200
  Remote Address: 35.244.167.217:443
  Referrer Policy: no-referrer-when-downgrade
▼Response Headers
  access-control-allow-origin: *
  alt-svc: clear
  cache-control: max-age=60
```

Have you seen that there is a property called "size"? It exactly equals 24, which is the page size. Let's try to change that to `size=220`, and send the request using the Python built-in library `requests`.

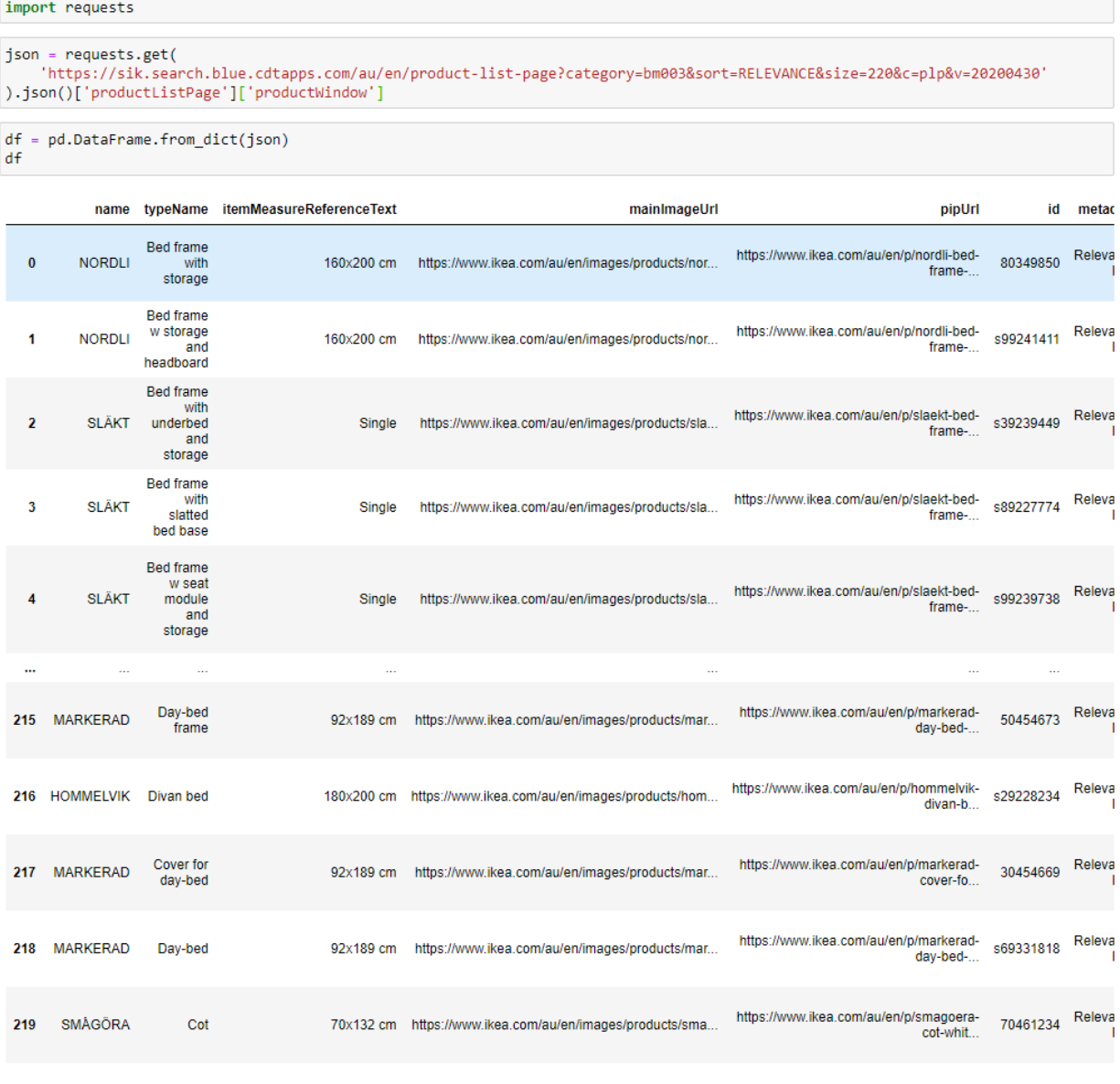
```
import requests
```

```
json = requests.get(
    'https://sik.search.blue.cdtapps.com/au/en/product-list-page?category=bm003&amp;sort=RELEVANCE&amp;size=220&amp;c=plp&
).json()['productListPage']['productWindow']
```

The reason why we cannot directly use Pandas this time is that the product list is not at the root level of the JSON object. Instead, it is at `root.productListPage.productWindow`.

After that, the request will convert the JSON object to a Python dictionary, so we can read it using Pandas now.

```
df = pd.DataFrame.from_dict(json)
```



```
import requests
```

```
json = requests.get(
    'https://sik.search.blue.cdtapps.com/au/en/product-list-page?category=bm003&sort=RELEVANCE&size=220&c=plp&v=20200430'
).json()['productListPage']['productWindow']
```

```
df = pd.DataFrame.from_dict(json)
df
```

	name	typeName	itemMeasureReferenceText	mainImageUrl	pipUrl	id	metad
0	NORDLI	Bed frame with storage	160x200 cm	https://www.ikea.com/au/en/images/products/nor...	https://www.ikea.com/au/en/p/nordli-bed-frame-...	80349850	Releva
1	NORDLI	Bed frame w storage and headboard	160x200 cm	https://www.ikea.com/au/en/images/products/nor...	https://www.ikea.com/au/en/p/nordli-bed-frame-...	s99241411	Releva
2	SLÄKT	Bed frame with underbed and storage	Single	https://www.ikea.com/au/en/images/products/sla...	https://www.ikea.com/au/en/p/slaekt-bed-frame-...	s39239449	Releva
3	SLÄKT	Bed frame with slatted bed base	Single	https://www.ikea.com/au/en/images/products/sla...	https://www.ikea.com/au/en/p/slaekt-bed-frame-...	s89227774	Releva
4	SLÄKT	Bed frame w seat module and storage	Single	https://www.ikea.com/au/en/images/products/sla...	https://www.ikea.com/au/en/p/slaekt-bed-frame-...	s99239738	Releva
...	...	...	...	...	...	...	
215	MARKERAD	Day-bed frame	92x189 cm	https://www.ikea.com/au/en/images/products/mar...	https://www.ikea.com/au/en/p/markerad-day-bed-...	50454673	Releva
216	HOMMELVIK	Divan bed	180x200 cm	https://www.ikea.com/au/en/images/products/hom...	https://www.ikea.com/au/en/p/hommelvik-divan-b...	s29228234	Releva
217	MARKERAD	Cover for day-bed	92x189 cm	https://www.ikea.com/au/en/images/products/mar...	https://www.ikea.com/au/en/p/markerad-cover-fo...	30454669	Releva
218	MARKERAD	Day-bed	92x189 cm	https://www.ikea.com/au/en/images/products/mar...	https://www.ikea.com/au/en/p/markerad-day-bed-...	s69331818	Releva
219	SMÅGÖRA	Cot	70x132 cm	https://www.ikea.com/au/en/images/products/sma...	https://www.ikea.com/au/en/p/smagoera-cot-whit...	70461234	Releva

220 rows × 10 columns

Summary

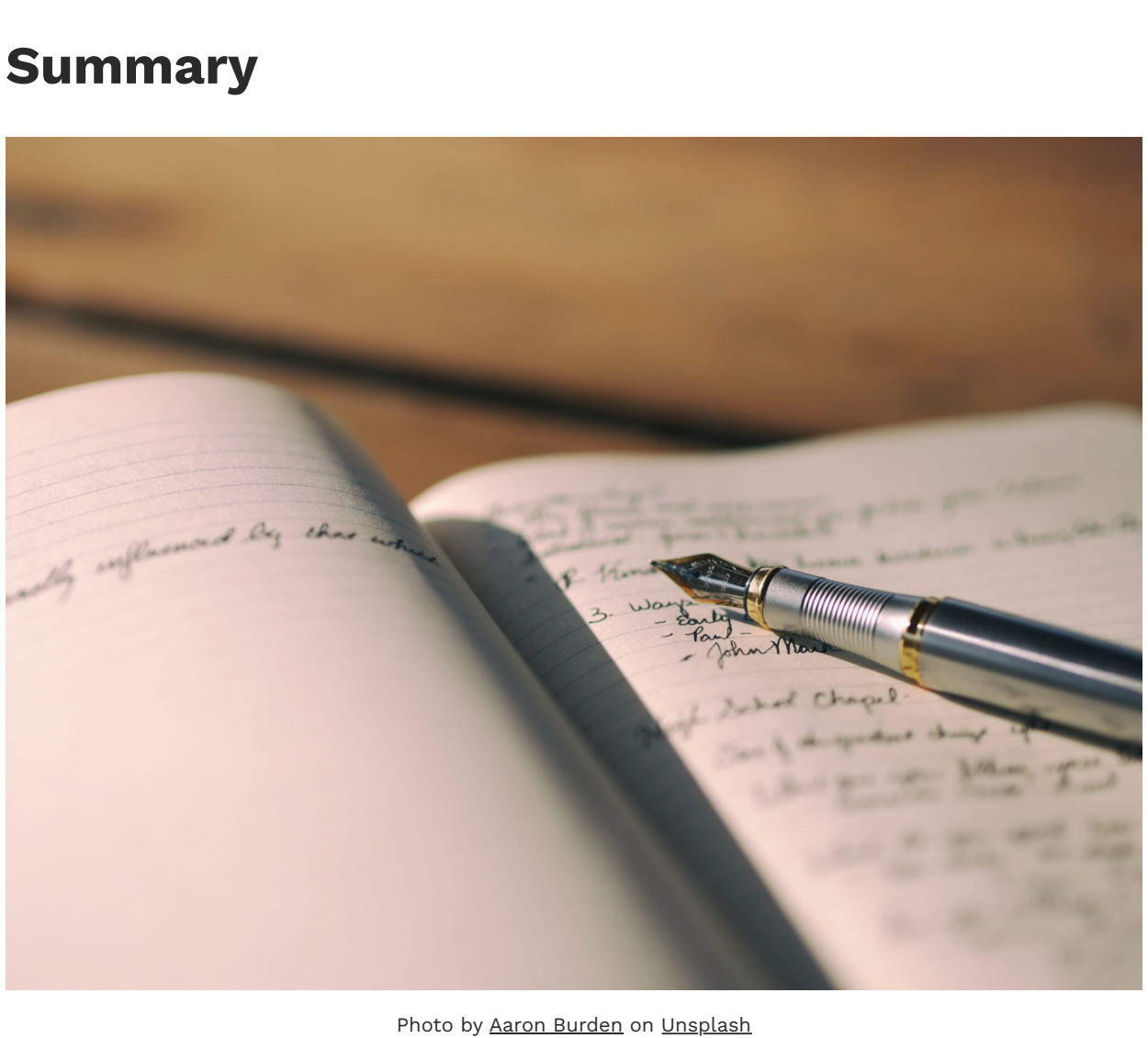


Photo by Aaron Burden on Unsplash

Isn't that quick and easy? You don't have to learn those web scraping libraries. But, of course, this approach is not going to be effective for all the websites, and that is why the web scraping libraries are still necessary. However, when there is an available API endpoint on the website, why not just find it and directly use it?

> **Join Medium with my referral link – Christopher Tao**

If you feel my articles are helpful, please consider joining Medium Membership to support me and thousands of other writers! (Click the link above)

Reference

[1] Wikipedia. XMLHttpRequest. https://en.wikipedia.org/wiki/XMLHttpRequest

• • •

WRITTEN BY

Christopher Tao

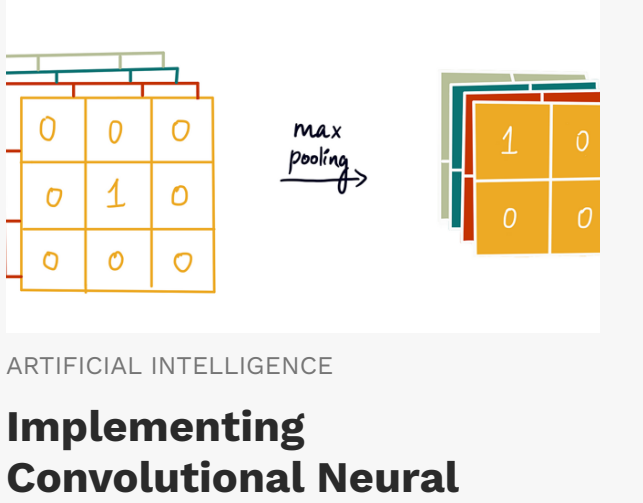
See all from Christopher Tao

Topics:

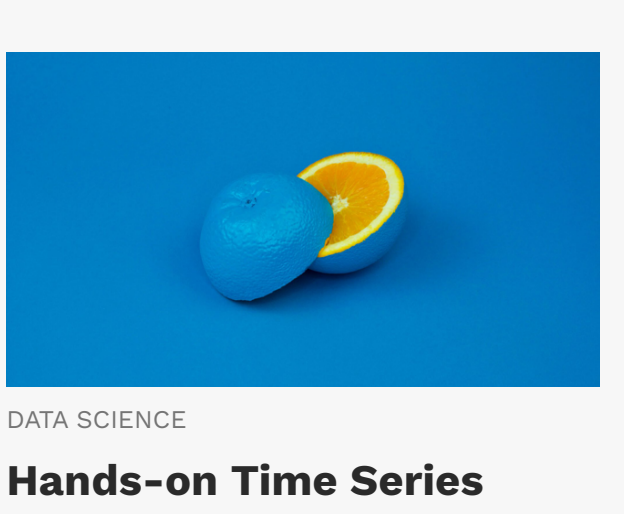
Covid-19 Data Data Analysis Data Science Python

Share this article: ○ ○ ○

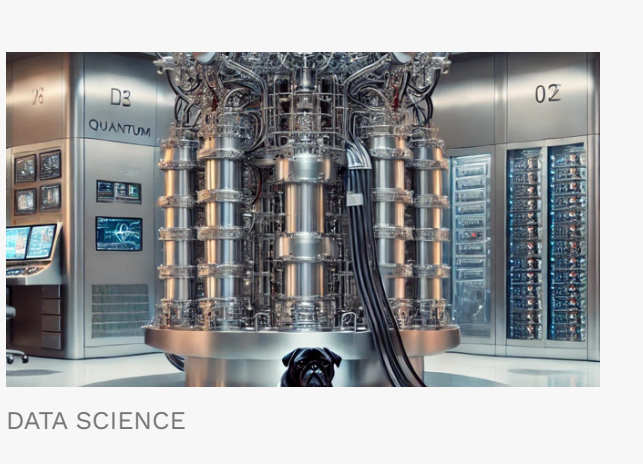
Related Articles



ARTIFICIAL INTELLIGENCE
Implementing Convolutional Neural Networks in TensorFlow
Step-by-step code guide to building a Convolutional Neural Network
Shreya Rao
August 20, 2024 • 6 min read

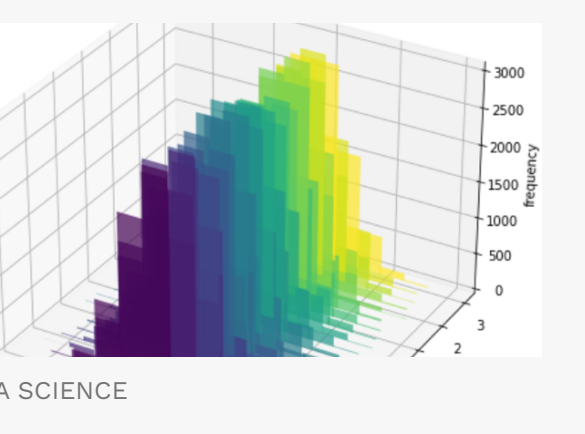


DATA SCIENCE
Hands-on Time Series Anomaly Detection using Autoencoders, with Python
Here's how to use Autoencoders to detect signals with anomalies in a few lines of…
Piero Paialunga
August 21, 2024 • 12 min read



DATA SCIENCE
Solving a Constrained Project Scheduling Problem with Quantum Annealing
Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)
Luis Fernando PÉREZ ARMAS, Ph.D.
August 20, 2024 • 28 min read

DATA SCIENCE
Back To Basics, Part Uno: Linear Regression and Cost Function
An illustrated guide on essential machine learning concepts
Shreya Rao
February 3, 2023 • 6 min read

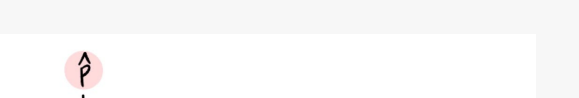


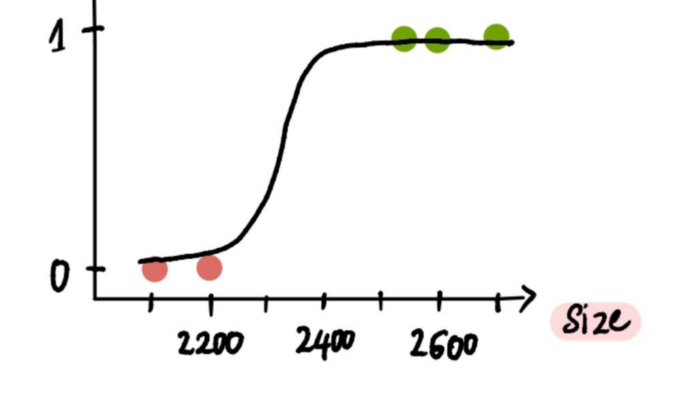
DATA SCIENCE
Must-Know in Statistics: The Bivariate Normal Projection Explained
Derivation and practical examples of this powerful concept
Luigi Battistoni
August 14, 2024 • 7 min read

DATA SCIENCE
How to Make the Most of Your Experience as a TDS Author
A quick guide to our resources and FAQ
TDS Editors
September 13, 2022 • 4 min read

DATA SCIENCE
Our Columns
Columns on TDS are carefully curated collections of posts on a particular idea or category…
TDS Editors
November 14, 2020 • 4 min read

DATA SCIENCE
Optimizing Marketing Campaigns with Budgeted Multi-Armed Bandits
With demos, our new solution, and a video
Vadim Arzamasov
August 16, 2024 • 10 min read





DATA SCIENCE

Back to Basics, Part Tres: Logistic Regression

An illustrated guide to everything you need to know about Logistic Regression

Shreya Rao

March 2, 2023 • 8 min read

towards
data science

Your home for data science and Al. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2025

Sign up to our newsletter

Your email*

First name*

Last name*

Job title*

Job level*

Please Select

Company name*

☐ I consent to receive newsletters and other communications from Towards Data Science publications.*

Subscribe Now